ЛАБОРАТОРНА РОБОТА № 2

Тема: Організація коду в мові Go. Пакети. Тестування

Мета: Розглянути організацію коду в мові Go: головнийпакет;додаткові пакети, зовнішні пакети. Область видимості ідентифікаторів пакета. Отримати практичні навички у створенні пакетів (одного, двохі більше),організувати тестування функцій пакета.

хід роботи

Завдання 1.

Реалізувати програму в одному вхідному файлі з трьомафункціями: знаходження мінімального значення з трьох елементів, обчисленнясередньогозначення трьох елементів і рішення рівняння першого порядку;

Виконання завдання:

Код програми:

```
package main
import "fmt"
// Min знаходить мінімальне значення з трьох елементів
func Min(a, b, c float64) float64 {
   min := a
   if b < min {
      min = b
   }
   if c < min {
      min = c
   }
   return min
}
```

```
// Average обчислює середнє значення трьох елементів
func Average(a, b, c float64) float64 {
    return (a + b + c) / 3.0
}
// SolveLinearEquation розв'язує рівняння першого порядку ax + b = 0
// Повертає x = -b/a
func SolveLinearEquation(a, b float64) (float64, error) {
    if a == 0 {
        return 0, fmt.Errorf("ділення на нуль: коефіцієнт a не може бути рівним нулю")
    }
    return -b / a, nil
}
func main() {
    // Приклад використання функцій
    fmt.Println("Мінімальне значення з (5, 2, 8):", Min(5, 2, 8))
    fmt.Println("Середнє значення (5, 10, 15):", Average(5, 10, 15))

    x, err := SolveLinearEquation(2, -4)
    if err != nil {
        fmt.Println("Помилка:", err)
    } else {
        fmt.Println("Розв'язок рівняння 2x - 4 = 0: x =", x)
    }
}
```

Результат виконання:

```
Мінімальне значення з (5, 2, 8): 2
Середнє значення (5, 10, 15): 10
Розв'язок рівняння 2x - 4 = 0: x = 2

Process finished with the exit code 0
```

Рис. 2.1

Завдання 2.

Розбити додаток з 2 вихідними файлами (у другійфайлвинестиреалізовані три функції);

Виконання завдання:

Код програми:

Файл functions.go

```
package main

import "fmt"

// Min знаходить мінімальне значення з трьох елементів
func Min(a, b, c float64) float64 {
   min := a
   if b < min {
      min = b
   }
   if c < min {
      min = c
   }
   return min
}

// Average обчислює середнє значення трьох елементів
func Average(a, b, c float64) float64 {
   return (a + b + c) / 3.0
}
// SolveLinearEquation розв'язує рівняння першого порядку ax + b = 0
// Повертає x = -b/a
func SolveLinearEquation(a, b float64) (float64, error) {
   if a == 0 {
      return 0, fmt.Errorf("ділення на нуль: коефіцієнт a не може бути рівним нулю")
   }
   return -b / a, nil
}
```

Файл main.go

```
package main
```

```go
import "fmt"

func main() {
    // Тестування функції Min
    fmt.Println("Мінімальне значення з (5, 2, 8):", Min(5, 2, 8))
    fmt.Println("Мінімальне значення з (-3, -1, -5):", Min(-3, -1, -5))
    fmt.Println("Мінімальне значення з (10, 10, 10):", Min(10, 10, 10))

    // Тестування функції Average
    fmt.Println("Середнє значення (5, 10, 15):", Average(5, 10, 15))
    fmt.Println("Середнє значення (1, 2, 3):", Average(1, 2, 3))
    fmt.Println("Середнє значення (-5, 0, 5):", Average(-5, 0, 5))

    // Тестування функції SolveLinearEquation
    x1, err1 := SolveLinearEquation(2, -4)
    if err1 != nil {
      fmt.Println("Помилка:", err1)
    } else {
      fmt.Println("Розв'язок рівняння 2x - 4 = 0: x =", x1)
    }

    x2, err2 := SolveLinearEquation(3, 6)
    if err2 != nil {
      fmt.Println("Помилка:", err2)
    } else {
      fmt.Println("Розв'язок рівняння 3x + 6 = 0: x =", x2)
    }

    x3, err3 := SolveLinearEquation(0, 5)
    if err3 != nil {
      fmt.Println("Помилка:", err3)
    } else {
      fmt.Println("Розв'язок рівняння 0x + 5 = 0: x =", x3)
    }
}
```

Результат виконання:

```
Мінімальне значення з (5, 2, 8): 2
Мінімальне значення з (-3, -1, -5): -5
Мінімальне значення з (10, 10, 10): 10
Середнє значення (5, 10, 15): 10
Середнє значення (1, 2, 3): 2
Середнє значення (-5, 0, 5): 0
Розв'язок рівняння 2x - 4 = 0: x = 2
Розв'язок рівняння 3x + 6 = 0: x = -2
Помилка: ділення на нуль: коефіцієнт a не може бути рівним нулю

Process finished with the exit code 0
```

Рис. 2.2

Завдання 3.

Реалізувати пакет (в пакет винести реалізовані три функції);

Виконання завдання:

Код програми:

Файл calc/calc.go:

```
package calc
import "fmt"
// Min знаходить мінімальне значення з трьох елементів
func Min(a, b, c float64) float64 {
    min := a
    if b < min {
        min = b
    }
    if c < min {
        min = c
    }
    return min
}
// Average обчислює середнє значення трьох елементів
func Average(a, b, c float64) float64 {
    return (a + b + c) / 3.0
}
// SolveLinearEquation розв'язує рівняння першого порядку ax + b = 0
// Повертає x = -b/a
func SolveLinearEquation(a, b float64) (float64, error) {
    if a == 0 {
```

```
        return 0, fmt.Errorf("ділення на нуль: коефіцієнт a не може бути рівним нулю")
    }
    return -b / a, nil
}
```

Файл main.go:

```
package main
import (
    "fmt"
    "ex3/calc"
)
func main() {
    // Тестування функції Min
    fmt.Println("Мінімальне значення з (5, 2, 8):", calc.Min(5, 2, 8))
    fmt.Println("Мінімальне значення з (-3, -1, -5):", calc.Min(-3, -1, -5))
    fmt.Println("Мінімальне значення з (10, 10, 10):", calc.Min(10, 10, 10))
    // Тестування функції Average
    fmt.Println("Середнє значення (5, 10, 15):", calc.Average(5, 10, 15))
    fmt.Println("Середнє значення (1, 2, 3):", calc.Average(1, 2, 3))
    fmt.Println("Середнє значення (-5, 0, 5):", calc.Average(-5, 0, 5))
    // Тестування функції SolveLinearEquation
    x1, err1 := calc.SolveLinearEquation(2, -4)
    if err1 != nil {
        fmt.Println("Помилка:", err1)
    } else {
        fmt.Println("Розв'язок рівняння 2x - 4 = 0: x =", x1)
    }
    x2, err2 := calc.SolveLinearEquation(3, 6)
    if err2 != nil {
        fmt.Println("Помилка:", err2)
    } else {
        fmt.Println("Розв'язок рівняння 3x + 6 = 0: x =", x2)
    }
    x3, err3 := calc.SolveLinearEquation(0, 5)
    if err3 != nil {
        fmt.Println("Помилка:", err3)
```

```
    } else {
        fmt.Println("Розв'язок рівняння 0x + 5 = 0: x =", x3)
    }
}
```

Результат виконання:

```
Мінімальне значення з (5, 2, 8): 2
Мінімальне значення з (-3, -1, -5): -5
Мінімальне значення з (10, 10, 10): 10
Середнє значення (5, 10, 15): 10
Середнє значення (1, 2, 3): 2
Середнє значення (-5, 0, 5): 0
Розв'язок рівняння 2x - 4 = 0: x = 2
Розв'язок рівняння 3x + 6 = 0: x = -2
Помилка: ділення на нуль: коефіцієнт a не може бути рівним нулю

Process finished with the exit code 0
```

Рис. 2.3

Завдання 4.

1. Додайте тести для вашого пакета;

Виконання завдання:

Код програми:

Файл calc/calc_test.go

```
package calc
import "testing"
// TestMin тестує функцію Min
func TestMin(t *testing.T) {
    tests := []struct {
        name     string
        a        float64
        b        float64
        c        float64
        expected float64
    }{
        {"Позитивні числа", 5, 2, 8, 2},
        {"Негативні числа", -3, -1, -5, -5},
        {"Однакові числа", 10, 10, 10, 10},
        {"Змішані числа", -5, 0, 5, -5},
        {"Дробові числа", 1.5, 2.3, 0.8, 0.8},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := Min(tt.a, tt.b, tt.c)
            if result != tt.expected {
                t.Errorf("Min(%f, %f, %f) = %f; очікується %f", tt.a, tt.b, tt.c, result, tt.expected)
            }
        })
    }
```

```
}

// TestAverage тестує функцію Average
func TestAverage(t *testing.T) {
    tests := []struct {
        name     string
        a        float64
        b        float64
        c        float64
        expected float64
    }{
        {"Позитивні числа", 5, 10, 15, 10.0},
        {"Послідовні числа", 1, 2, 3, 2.0},
        {"Змішані числа", -5, 0, 5, 0.0},
        {"Однакові числа", 3, 3, 3, 3.0},
        {"Дробові числа", 1.5, 2.5, 3.5, 2.5},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result := Average(tt.a, tt.b, tt.c)
            if result != tt.expected {
                t.Errorf("Average(%f, %f, %f) = %f; очікується %f", tt.a, tt.b, tt.c, result, tt.expected)
            }
        })
    }
}

// TestSolveLinearEquation тестує функцію SolveLinearEquation
func TestSolveLinearEquation(t *testing.T) {
    tests := []struct {
        name        string
        a           float64
        b           float64
        expectedX   float64
        expectError bool
    }{
        {"Нормальне рівняння 1", 2, -4, 2.0, false},
        {"Нормальне рівняння 2", 3, 6, -2.0, false},
        {"Нормальне рівняння 3", -2, 4, 2.0, false},
        {"Ділення на нуль", 0, 5, 0, true},
        {"Дробові коефіцієнти", 0.5, -1, 2.0, false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result, err := SolveLinearEquation(tt.a, tt.b)
            if tt.expectError {
                if err == nil {
                    t.Errorf("SolveLinearEquation(%f, %f) очікувала помилку, але отримала результат %f",
tt.a, tt.b, result)
                }
```

```
        } else {
          if err != nil {
            t.Errorf("SolveLinearEquation(%f, %f) повернула помилку: %v", tt.a, tt.b, err)
          }
          if result != tt.expectedX {
            t.Errorf("SolveLinearEquation(%f, %f) = %f; очікується %f", tt.a, tt.b, result, tt.expectedX)
          }
        }
      })
    }
}

// BenchmarkMin бенчмарк для функції Min
func BenchmarkMin(b *testing.B) {
    for i := 0; i < b.N; i++ {
      Min(5, 2, 8)
    }
}

// BenchmarkAverage бенчмарк для функції Average
func BenchmarkAverage(b *testing.B) {
    for i := 0; i < b.N; i++ {
      Average(5, 10, 15)
    }
}

// BenchmarkSolveLinearEquation бенчмарк для функції SolveLinearEquation
func BenchmarkSolveLinearEquation(b *testing.B) {
    for i := 0; i < b.N; i++ {
      _, _ = SolveLinearEquation(2, -4)
    }
}
```

Результат виконання:

```
✓ 18 tests passed  18 tests total, 54 ms
=== RUN   TestMin
=== RUN   TestMin/Позитивні_числа
--- PASS: TestMin/Позитивні_числа (0.00s)
=== RUN   TestMin/Негативні_числа
--- PASS: TestMin/Негативні_числа (0.00s)
=== RUN   TestMin/Однакові_числа
--- PASS: TestMin/Однакові_числа (0.00s)
=== RUN   TestMin/Змішані_числа
--- PASS: TestMin/Змішані_числа (0.00s)
=== RUN   TestMin/Дробові_числа
--- PASS: TestMin/Дробові_числа (0.00s)
--- PASS: TestMin (0.00s)
=== RUN   TestAverage
=== RUN   TestAverage/Позитивні_числа
--- PASS: TestAverage/Позитивні_числа (0.00s)
=== RUN   TestAverage/Послідовні_числа
--- PASS: TestAverage/Послідовні_числа (0.00s)
=== RUN   TestAverage/Змішані_числа
--- PASS: TestAverage/Змішані_числа (0.00s)
=== RUN   TestAverage/Однакові_числа
--- PASS: TestAverage/Однакові_числа (0.00s)
=== RUN   TestAverage/Дробові_числа
--- PASS: TestAverage/Дробові_числа (0.00s)
--- PASS: TestAverage (0.00s)
=== RUN   TestSolveLinearEquation
=== RUN   TestSolveLinearEquation/Нормальне_рівняння_1
--- PASS: TestSolveLinearEquation/Нормальне_рівняння_1 (0.00s)
=== RUN   TestSolveLinearEquation/Нормальне_рівняння_2
--- PASS: TestSolveLinearEquation/Нормальне_рівняння_2 (0.00s)
=== RUN   TestSolveLinearEquation/Нормальне_рівняння_3
--- PASS: TestSolveLinearEquation/Нормальне_рівняння_3 (0.00s)
=== RUN   TestSolveLinearEquation/Ділення_на_нуль
--- PASS: TestSolveLinearEquation/Ділення_на_нуль (0.00s)
=== RUN   TestSolveLinearEquation/Дробові_коефіцієнти
--- PASS: TestSolveLinearEquation/Дробові_коефіцієнти (0.00s)
--- PASS: TestSolveLinearEquation (0.00s)
PASS
```

Рис. 2.4

Висновок

У ході лабораторної роботи було реалізовано програму мовою Go для виконання обчислень та розв’язання лінійного рівняння. Програму було послідовно організовано: від одного файлу до використання окремого пакета. Також було виконано модульне тестування, яке підтвердило правильність роботи всіх функцій. У результаті було засвоєно основи роботи з пакетами та тестуванням у Go.

Контрольні питання

1. Як організована структура коду в мові Go?

Код у Go організовується за допомогою пакетів. Програма складається з одного або кількох пакетів, які можуть містити декілька файлів.

2. Що таке головний пакет?

Головний пакет — це пакет, з якого починається виконання програми.

3. Яке ім’я головного пакету?

Головний пакет має ім’я: **main**

4. Скільки файлів знаходиться у головному пакеті?

У головному пакеті може бути один або декілька файлів.

5. Де розміщуються пакети?

Пакети розміщуються у директоріях (папках), де кожна папка відповідає окремому пакету.

6. Як організувати тестування пакетів?

Тестування організовується за допомогою файлів з суфіксом _test.go, використовуючи пакет testing. Тести запускаються командою: **go test**